Multimodal Large Model + Robotic Arm Gripping（Text Version）

Before running the function, you need to close the App and large programs. For the closing method, refer to [4. Preparation] - [1. Manage APP control services].

1. Function Description

After the program runs, input robotic arm gripping action commands through the terminal. Based on the type of object to be gripped, the large model will start external programs to control the robotic arm to grip the target object.

2. Startup

After entering the Docker container, enter the following commands in the Docker terminal.

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and input the following command:

```
ros2 run text_chat text_chat
```

The robotic arm gripping has the following action commands:

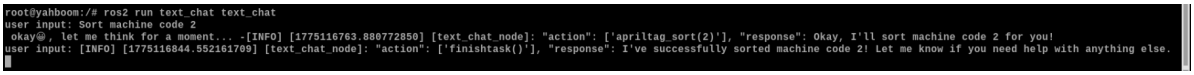
- **Grip by machine code ID**: Sort machine codes 1/2/3/4
- **Grip other objects**: Grip the small yellow duck on the desktop, grip the blue cuboid on the desktop

2.1. Grip by Machine Code ID

2.1.1. Startup

Input in the terminal that started text_chat:

```
Sort machine code 2
```



Then two terminals will open, starting the machine code recognition program and gripping program respectively. After recognizing machine code 2, the gripping program will control the robotic arm to descend and grip machine code 2, then place it at the set position, and finally return to the machine code recognition posture to prepare for the next gripping. This continues until no more machine code 2 is recognized, then it reports back to the large model and the task ends.

2.1.2. Task Planning

Plan the action function apriltag_sort(2), where the parameter 2 represents the ID of the machine code to be gripped.

2.1.3. Core Code Analysis

apriltag_sort function, source code located at:
`LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def apriltag_sort(self, target_id):  # Grip machine code
    #self.check_apriltag_sort()
    target_idf = float(target_id)
    #Start gripping program
    cmd1 = "ros2 run largemodel_arm grasp
    #Start machine code recognition program, passing parameter target_id
    cmd2 = f"ros2 run largemodel_arm apriltag_sort --ros-args -p target_id:=
{target_idf:.1f}"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop_apritag",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=apriltag_sort",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )
```

Gripping program grasp, source code path at:
`LargeModel_ws/src/largemodel_arm/largemodel_arm/grasp.py`

```
#Create subscriber, subscribe to position information topic
self.sub = self.create_subscription(AprilTagInfo,'PosInfo',self.pos_callback,1)
#Position information topic callback function
def pos_callback(self,msg):
    pos_x = msg.x
    pos_y = msg.y
    pos_z = msg.z
    self.cur_tagId = msg.id
    if pos_z!=0.0:
        print("xyz id : ",pos_x,pos_y,pos_z,self.cur_tagId)
        #Start coordinate system transformation
        camera_location = self.pixel_to_camera_depth((pos_x,pos_y),pos_z)
        PoseEndMat = np.matmul(self.EndToCamMat,
self.xyz_euler_to_mat(camera_location, (0, 0, 0)))
        EndPointMat = self.get_end_point_mat()
        WorldPose = np.matmul(EndPointMat, PoseEndMat)
        #Convert rotation transformation matrix to xyz and Euler angles to get
target pose
```

```
        pose_T, pose_R = self.mat_to_xyz_euler(WorldPose)
        pose_T[0] = pose_T[0] + self.x_offset
        pose_T[1] = pose_T[1] + self.y_offset
        pose_T[2] = pose_T[2] + self.z_offset
        print("pose_T: ",pose_T)
        if self.grasp_flag == True :
            print("Take it now.")
            self.grasp_flag = False
            #Pass target position information to grasp function for inverse
kinematics to get each robotic arm angle value
            threading.Thread(target=self.grasp, args=(pose_T,)).start()
#Call inverse kinematics service, pass target position, get values for servos 1-
5
def grasp(self,pose_T):
    print("-------------------------------------------------")
    print("pose_T: ",pose_T)

    request = Kinemarics.Request()
    request.tar_x = pose_T[0]
    request.tar_y = pose_T[1] + 0.01
    request.tar_z = pose_T[2] + 0.01
    request.kin_name = "ik"
    request.roll = -1.0 #((2.5*request.tar_y*100)-207.5)*(math.pi / 180)

    # print("calcutelate_request: ",request)

    try:
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
        response = future.result()
        print("calcutelate_response: ",response)
        joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
        joints[0] = response.joint1 #response.joint1
        joints[1] = response.joint2
        joints[2] = response.joint3
```

Machine code recognition program apriltag_sort, source code path at: `LargeModel_ws/src/largemodel_arm/largemodel_arm/apriltag_sort.py`,

```
#Get target sorting id through command line arguments
self.declare_parameter('target_id', 0.0)
self.TargetID =
int(self.get_parameter('target_id').get_parameter_value().double_value)

#Create publisher, publish position information topic
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)

#Image callback function
def TagDetect(self,color_frame,depth_frame):
    #rgb_image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    result_image = np.copy(rgb_image)
    #depth_image
    depth_image = self.depth_bridge.imgmsg_to_cv2(depth_frame, encoding[1])
    depth_to_color_image = cv.applyColorMap(cv.convertScaleAbs(depth_image,
alpha=1.0), cv.COLORMAP_JET)
```

```
    frame = cv.resize(depth_image, (640, 480))
    depth_image_info = frame.astype(np.float32)
    #Machine code recognition
    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)
    tags = sorted(tags, key=lambda tag: tag.tag_id)
    draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0,
255, 0))
    key = cv2.waitKey(10)
    if self.count==True:
        if (time.time() - self.start_time)>10:
            self.pubPos_flag = True
            self.count = False
    if key == 32:
        self.pubPos_flag = True
    if len(tags) > 0 :
        for i in range(len(tags)):
            center_x, center_y = tags[i].center
            cur_id = tags[i].tag_id
            cv.circle(depth_to_color_image,(int(center_x),int(center_y)),1,
(255,255,255),10)
            #If current id is target sorting id, it means found, so
self.detect_flag value is True, and record current array index for subsequent
value retrieval
            if self.pubPos_flag == True and cur_id == self.TargetID:
                self.detect_flag = True
                print("Found the target.")
                self.index = i
                center_x, center_y = tags[self.index].center
                cv.circle(result_image, (int(center_x),int(center_y)), 10,
(0,210,255), thickness=-1)

                pos = AprilTagInfo()
                #Get xy coordinates of target machine code, center point depth
value and id
                pos.id = tags[self.index].tag_id
                pos.x = center_x
                pos.y = center_y
                pos.z = depth_image_info[int(center_y),int(center_x)]/1000
                vx = int(tags[self.index].corners[0][0]) -
int(tags[self.index].corners[1][0])
                vy = int(tags[self.index].corners[0][1]) -
int(tags[self.index].corners[1][1])
                target_joint5 = compute_joint5(vx,vy)
                if pos.z>0:
                    self.joint5.data = int(target_joint5)
                    self.TargetJoint5_pub.publish(self.joint5)
                    #Publish position information topic data message
                    self.pos_info_pub.publish(pos)
                    self.pubPos_flag = False
                else:
                    print("Invalid distance.")
         #Machine code not found
        if  self.detect_flag == False and self.TargetID!=0 and
self.pubPos_flag==True:
            self.pubPos_flag = False
            print("Did not find the target.")
```

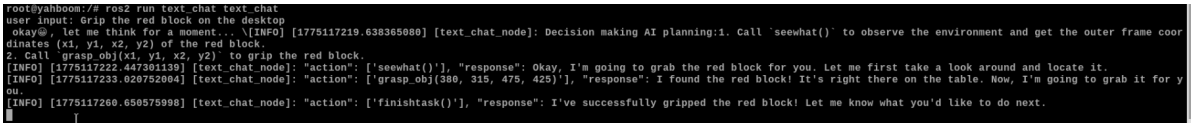
```
            self.largemodel_arm_done_pub.publish(
String(data='apriltag_sort_done'))
            self.TargetID = 0
```

2.2. Grip Other Objects

2.2.1. Startup

Input in the terminal that started text_chat:

```
Grip the red block on the desktop
```



The program will first take a photo to determine the position of the small yellow duck, then open two terminals, starting the KCF tracking positioning program and object gripping program respectively. After calculation, the robotic arm will descend to grip the small yellow duck and finally return to the initial posture.

2.2.2. Task Planning

1. Call seehat() to check the position of the small yellow duck;
2. Call grasp_obj(x1,y1,x2,y2) gripping function, where x1,y1,x2,y2 represent the top-left and bottom-right corner coordinates of the small yellow duck's outer bounding box.

2.2.3. Core Code Analysis

The seehat() function was analyzed in the previous tutorial [2. Multimodal Visual Understanding] and will not be repeated here. Let's mainly look at the grasp_obj function, source code located at: `LargeModel_ws/src/largemodel/largemodel/action_service.py`

```
def grasp_obj(self, x1, y1, x2, y2):
    """
    Grip object
    x1,y1,x2,y2: Object outer bounding box coordinates
    """
    #Arm.Arm_serial_servo_write6(90,120,12,20,90,30,1000)
    #self.check_close_grasp_obj()
    #Object gripping program
    cmd1 = "ros2 run largemodel_arm KCF_Grap_Move"
    #KCF tracking positioning program
    cmd2 = "ros2 run largemodel_arm ALM_KCF_Tracker"
    # cmd3 = "ros2 run --prefix 'gdb -ex run --args' M3Pro_KCF
ALM_KCF_Tracker_Node"
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=ALM_KCF_Tracker",
            "--",
            "bash",
            "-c",
            f"{cmd2}; exec bash",
        ]
    )
    time.sleep(5.0) #Wait for ALM_KCF_Tracker to start up
```

```
    subprocess.Popen(
        [
            "gnome-terminal",
            "--title=grasp_desktop",
            "--",
            "bash",
            "-c",
            f"{cmd1}; exec bash",
        ]
    )
    time.sleep(2.0)
    if self.stack_flag == True:
        self.get_logger().info('Publish the stack_step topic...')
        step_ = Int16()
        step_.data = self.step
        self.step_pub.publish(step_)
        self.step = self.step + 1
    #Publish outer bounding box coordinate information topic data,
ALM_KCF_Tracker node will subscribe to this topic
    x1 = int(x1)
    y1 = int(y1)
    x2 = int(x2)
    y2 = int(y2)
    self.object_position_pub.publish(Int16MultiArray(data=[x1, y1, x2, y2]))
```

KCF_Grap_Move program, source code location at:
`LargeModel_ws/src/largemodel_arm/largemodel_arm/KCF_Grap_Move.py`

```
#Import library, this library's path is
LargeModel_ws/src/largemodel_arm/largemodel_arm/Dofbot_Track.py
from largemodel_arm.Dofbot_Track import *

#Create robotic arm tracking gripping object
self.dofbot_tracker = DofbotTrack()

#Create subscriber, subscribe to /pos_xyz topic
self.pos_sub = Subscriber(self,Position,'/pos_xyz')

#Callback function
def TrackAndGrap(self,position):
    center_x, center_y = position.x,position.y
    self.cur_distance = position.z
    if self.done == True:
        self.get_current_end_pos()
        self.get_logger().info('-*-*-*-*-*-*-*-*-*-*')
        print("self.CurEndPos: ",self.CurEndPos)
        #Call external library function Clamping

 self.dofbot_tracker.Clamping(center_x,center_y,self.cur_distance,self.CurEndPos
)
        self.done = False

#View Clamping function in Dofbot_Track.py
def Clamping(self,cx,cy,cz,CurEndPos):
    self.CurEndPos = CurEndPos
    #Start coordinate system transformation
    camera_location = self.pixel_to_camera_depth((cx,cy),cz)
```

```
    print("cz: ",cz)
    PoseEndMat = np.matmul(self.EndToCamMat,
self.xyz_euler_to_mat(camera_location, (0, 0, 0)))
    EndPointMat = self.get_end_point_mat()
    WorldPose = np.matmul(EndPointMat, PoseEndMat)
    #Convert rotation transformation matrix to xyz and Euler angles
    pose_T, pose_R = self.mat_to_xyz_euler(WorldPose)
    pose_T[0] = pose_T[0] + self.x_offset
    pose_T[1] = pose_T[1] + self.y_offset
    pose_T[2] = pose_T[2] + self.z_offset
    print("pose_T: ",pose_T)
    #Create inverse kinematics service object and assign the calculated pose_T
representing target position to it
    request = Kinemarics.Request()
    if pose_T[0]>0:
        request.tar_x = pose_T[0] +0.01
    else:
        request.tar_x = pose_T[0] - 0.01
    request.tar_z = pose_T[2]
    request.tar_y = pose_T[1] + request.tar_z*0.2
    request.kin_name = "ik"
    request.roll =  -1.0
    print("calcutelate_request: ",request)

    try:
        #Call inverse kinematics service, calculate to get servo values 1-4
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=10.0)
        response = future.result()
        joints = [0.0, 0.0, 0.0, 0.0, 0.0,0.0]
        joints[0] = response.joint1 #response.joint1
        joints[1] = response.joint2
        joints[2] = response.joint3
        if response.joint4>90:
            joints[3] = 90.0
        else:
            joints[3] = response.joint4
        joints[4] = 90.0
        joints[5] = 20.0
        print("compute_joints: ",joints)
```

ALM_KCF_Tracker program, source code location at:
`LargeModel_ws/src/largemodel_arm/largemodel_arm/ALM_KCF_Tracker.py`

```
#Create publisher, publish object position information
self.pub_pos = self.create_publisher(Position, "/pos_xyz", 10)
#Create subscriber, subscribe to object outer bounding box information
self.xy_subscription =
self.create_subscription(Int16MultiArray,'corner_xy',self.GetXYCallback,qos_prof
ile=1)

#Callback function
def GetXYCallback(self,msg):
    print("msg: ",msg.data)
    print(msg.data[0])
    print(msg.data[1])
    print(msg.data[2])
```

```
    print(msg.data[3])
    #Assign self.Roi_init value to topic data message data
    self.Roi_init = (msg.data[0],msg.data[1],msg.data[2],msg.data[3])
    self.Track_state = 'identify'
    self.get_xy = True
    self.rect = [msg.data[0],msg.data[1],msg.data[2],msg.data[3]]

#In process function, initialize KCF based on self.Roi_init value
if self.gTracker_state == True:
    Roi = (self.Roi_init[0], self.Roi_init[1], self.Roi_init[2] -
self.Roi_init[0], self.Roi_init[3] - self.Roi_init[1])
    self.compute_roi = True
    print("Roi: ",Roi)
    self.gTracker = Tracker(tracker_type=self.tracker_type)
    self.gTracker.initWorking(rgb_img, Roi)
    self.gTracker_state = False
#KCF tracking, get object center point information
rgb_img, (targBegin_x, targBegin_y), (targEnd_x, targEnd_y) =
self.gTracker.track(rgb_img)
center_x = targEnd_x / 2 + targBegin_x / 2
center_y = targEnd_y / 2 + targBegin_y / 2
self.cx = center_x
self.cy = center_y
#In kcfTrack image callback function
#Assign calculated center point coordinates to center_x, center_y
center_x, center_y = self.cx,self.cy
cv2.circle(depth_to_color_image,(int(center_x),int(center_y)),1,
(255,255,255),10)
#Calculate center point depth distance information
self.cur_distance = depth_image_info[int(center_y),int(center_x)]/1000.0
dist = round(self.cur_distance,3)
dist = 'dist: ' + str(dist) + 'm'
cv.putText(result_frame, dist,  (30, 30), cv.FONT_HERSHEY_SIMPLEX, 1.0, (255, 0,
0), 2)
pos = Position()
pos.x = center_x
pos.y = center_y
pos.z = self.cur_dis  tance
Track_pos = Position()
Track_pos.x = center_x
Track_pos.y = center_y
#Publish position information topic data
self.pub_track_pos.publish(Track_pos)
```